Name: ALA-AN, PRINCESS BEVERLY R.
Section: BN01 Date: *August 9, 2025*
Course: PROGRAMMING LANGUAGES

I. **Give more strings** in L aside from w = 01010101. Reg Exp for L = ***(01)*+(10)*+0(10)*+1(01)****

1. ***1010101010*** **matches (10)***

2. ***0101010*** **matches 0(10)***

3. ***101010101*** **matches 1(01)***

4. **“” matches (01)*+(10)*+0(10)*+1(01)***

5. ***0101*** **matches (01)***

6. ***1010*** **matches (10)***

7. ***010*** **matches 0(10)***

8. ***101*** **matches 1(01)***

9. 1 matches 1(01)*

10. 0 matches 0(10)*

II. **RE = 1*00*1(0+1)***

Task:

i. Construct one full valid string that matches the regular expression by explicitly applying each operator ({*}, {+}).

1. Indicate clearly which parts of the regular expression.

Answer: ***11100111***

where:

111 matches 1* in 1*00*1(0+1)*

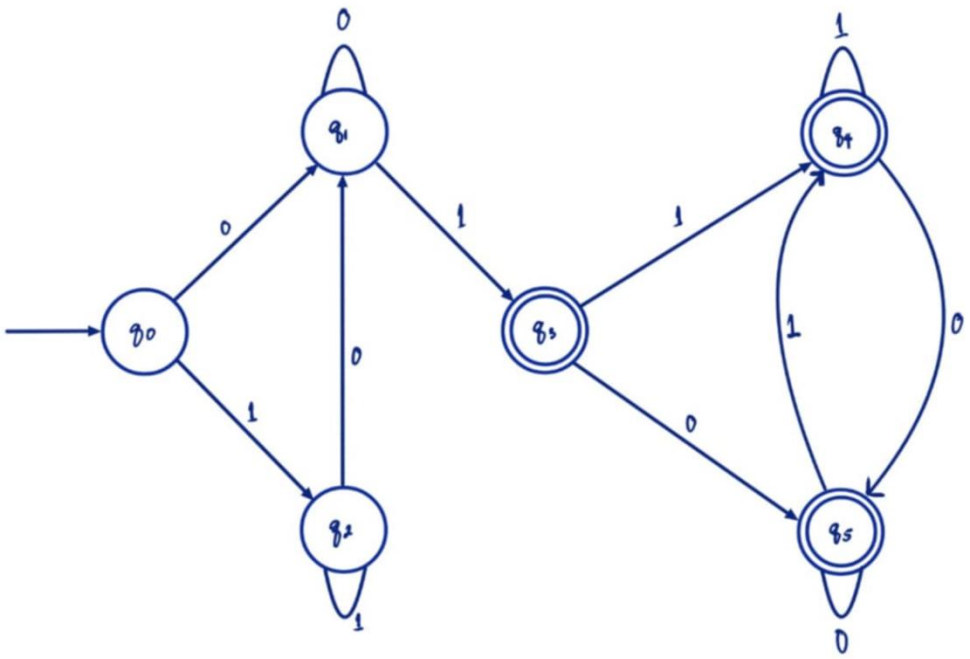
0 matches 0 in 1*0*0*1(0+1)*

0 matches 0* in 1*00*1(0+1)*

1 matches 1 in 1*00*1(0+1)*

11 matches 1* in 1*00*1(0+1)*

ii. Draw the DFA that accepts the same string:



1. Label all states and transitions,
2. Identify the start and accepting state(s),

State/Input	0	1
->q_0	q_1	q_2
q_1	q_1	*q_3
q_2	q_1	q_2
*q_3	*q_5	*q_4
*q_4	*q_5	*q_4
*q_5	*q_5	*q_4

q_0 is the starting state.

*q_3, *q_4, and *q_5 are the accepting states.

3. Make sure the DFA accepts the constructed string in Question #1

Transition Equation Guide:
δ(current state, input symbol) = next state

*String: **11100111***

$\delta(q_0, 1) = q_2$
$\delta(q_2, 1) = q_2$
$\delta(q_2, 1) = q_2$
$\delta(q_0, 0) = q_1$
$\delta(q_0, 0) = q_1$
$\delta(q_0, 1) = q_3$
$\delta(q_5, 1) = q_5$
$\delta(q_5, 1) = q_5$

The string *11100111* is a valid string and is accepted by the DFA.

III. **Machine Code Translation Task**

Based on the sample CFG production rules presented in the project, consider the context-free language that generates the following variable declaration and assignment:

x: int
x = 100

Task: Write the equivalent machine-level code for this statement using an assembly language of your choice (e.g., x86, MIPS, or ARM), based on your understanding of instruction-level representation from your *Computer Architecture and Organization course*.

```
section .bss
    x   resd    1               ; declaring x and reserving 32-bit integer

section .text
    global _start               ; actualy instructions starts here

_start:
    mov     dword [x], 100      ; store 100 in x

    mov     eax, 1              ; syscall: exit
    xor     ebx, ebx            ; exit code 0
    int 0x80
; this is run using NASM 64-bit online compiler suited for 32-bit Linux
```